01 · PRODUCT THESIS

ContinuityMap at a glance

A transparent planning workspace for understanding declared dependencies between essential services.

Electricity, water, hospitals, communications, transport, and emergency response rely on other services and resources. The information is often spread across continuity plans, equipment lists, service descriptions, and exercise notes. During an exercise, teams may have to piece together who relies on whom, whether a backup is confirmed, and where each claim came from.

ContinuityMap puts those recorded relationships in one map. A planner can select a service, see what it relies on and what relies on it, check the evidence, and run the same what-if exercise again. Only relationships a human has marked verified affect the calculated result. Suggested links and open questions stay separate.

The result is a planning aid, not a forecast. It follows the recorded links and stated backup rules. It does not predict physical damage, actual power or water flow, how long a backup lasts, when service returns, or whether an organization is ready. If those facts are missing, the system leaves them unknown.

Example: when the fictional Northbank Substation Alpha is disrupted, water treatment remains available in the exercise because a verified rule says either Alpha or the Civic Solar Microgrid can supply it. A hospital is marked unavailable because its required health-records path runs through a data center powered by Alpha. This is what the demo's records say; it is not a prediction about a real hospital.

- A fictional Northbank network with 43 items, 80 relationship records, and three repeatable exercises.
- Human approval is required before a proposed relationship can affect an official scenario result.
- The demo can run locally. Developers can use PostgreSQL, FastAPI, React, NetworkX, and optional Ollama.

In the Alpha exercise, 17 entries are marked unavailable (the substation plus 16 downstream entries); two open questions are shown for review, not counted as failures.

02 · PROBLEM & RESEARCH

A coordination problem with real policy context

The challenge is maintaining a shared view of dependencies and evidence across organizational boundaries.

Resilience planning involves service owners, public authorities, utilities, hospitals, transport teams, suppliers, and emergency groups. One team may own a plan, another a facility, and another the communications service everyone uses. A useful shared map therefore needs more than lines: it needs a clear arrow direction, a named owner, supporting evidence, and a date when someone last checked the claim.

Public guidance points to the same planning need. NATO's 2026 resilience baseline discusses dependencies and cascading effects across essential sectors. FEMA asks planners to consider which functions support or depend on other functions. CISA/FEMA cyber-incident guidance asks planners to look upstream and downstream. NIST SP 800-34 is useful for information-system recovery, though it does not cover the whole cross-sector problem.

A 2024 research review found that infrastructure studies use different dependency labels and can lose track of which way a relationship points. ContinuityMap makes the direction visible: provider → service that depends on it. This is a practical design choice, not a claim that one universal research standard exists.

Existing resilience platforms already offer maps, dashboards, and dependency features. For example, Everbridge publicly describes Bridge tools for mapping services and dependencies, finding gaps, setting impact tolerances, and linking plans to response workflows. ContinuityMap should not claim those basic features are new. Its early idea is to make each relationship and its review status easy to inspect, then show repeatable results from only owner-confirmed links. Buyer comparisons are still needed.

- Likely initial users: resilience offices, utilities, hospitals, airports, infrastructure consultancies, and emergency planning teams.
- Likely customer problem: cross-team preparation and verification of dependency-aware exercises.
- Primary limitation: accurate, current dependency data is customer-specific and often hard to assemble.

Research links, source limits, and software license notes are collected in docs/SOURCES.md. Competitor fit and customer demand still need interviews and product comparisons.

03 · PRODUCT & WORKFLOW

From inventory to owner-reviewed action

The core workflow keeps the graph and its evidence in the same place.

The inventory lists services, facilities, equipment, and backup resources. Every relationship records what supplies what, the type of support, a source quote and reference, its review state, an owner, any stated conditions, and the date it was last checked. A planner can select an item on the map, inspect its relationships and evidence, filter by review state, and edit a record.

There are four review states. Verified means a person approved the claim and it can affect an exercise. Proposed means a possible link is waiting for review. Unknown means the source leaves a question open. Rejected means a reviewer declined the claim. Proposed, unknown, and rejected links do not make a service unavailable in the calculation.

A scenario starts with one or more resources chosen as unavailable. The app follows verified relationships, shows each result and its evidence path, and lists open questions separately. Users can edit the graph, rerun the same scenario, and export an impact report, a gap report, or a human-review checklist.

The Northbank demo covers electricity, water, hospitals, clinics, communications, emergency operations, transport, shelters, data services, fuel, and backup resources. All names and relationships are invented; it is not a map of a real city or national infrastructure.

- Load a JSON file and check for missing or conflicting relationship details.
- Select a scenario or enter a custom disruption set.
- Review results, open questions, and owner actions; export PDF reports.

A scenario can only be as complete as the relationship records people have collected, checked, and kept current.

04 · DATA MODEL & SIMULATION

A simple graph with explicit redundancy rules

Only verified, directed relationships determine official impact status.

Each arrow points from a provider to the service that relies on it. A dependent service can have one or more groups of required providers. ANY means at least one provider in that group is enough—for example, utility power or a generator. ALL means every provider is needed—for example, power and network access. When a service has multiple groups, each group must be satisfied.

The calculation starts with the resource or resources selected as disrupted. It checks verified groups and adds a service if one of its required groups is no longer met. Then it checks again, because newly unavailable services may support other services. It stops when another full check adds nothing. Cycles are flagged for review, but the repeated check cannot run forever.

Proposed and unknown relationships are handled as review questions. If their provider is disrupted, the app may flag the dependent for an owner to check, but it does not mark it unavailable because of that unverified link. Rejected links are excluded. The validator also checks missing nodes, duplicate or conflicting relationships, unsupported assumptions, and missing review dates.

The engine does not model capacity, flow, load shedding, time-to-failure, restoration time, physical damage, shared causes, or probability. If the owner has not supplied a value, it stays unknown. The result means only that a declared requirement is or is not met in this exercise.

- Substation Alpha unavailable: 17 unavailable entries, including the starting substation.
- East water pumping loses utility and backup power: 20 unavailable nodes.
- Regional communications outage: 9 unavailable nodes; independent radio remains available.

In the Alpha exercise, 2 possible paths are shown for review. They are not counted as unavailable services.

05 · AI & GOVERNANCE

AI proposes; owners decide

Local document extraction supports review without becoming an authority over the graph.

A user can upload an invented TXT, Markdown, CSV, PDF, or DOCX plan. The app looks for names already in the graph and asks an optional local Ollama model to suggest who depends on whom, with a supporting quote. The names must match existing graph items, and the quote must appear in the uploaded text. If a model is not configured or its output is unusable, the app uses a clearly labeled rule-based method.

Every extracted link starts as proposed and keeps its quote and document reference. A service owner can correct the link, add missing details, verify it, or reject it. Until a person verifies it, the link cannot change an official exercise result.

The continuity assistant starts from the calculated result, relationship evidence, open questions, recovery resources, and written exercise actions. It may draft a summary and suggested actions, but the user must review them. Citations must point to known evidence references. If the model response cannot be checked, the app uses a structured evidence summary instead. The assistant does not change service status or a recovery plan.

These checks reduce some AI risks but do not guarantee that an AI summary is correct. A valid citation ID does not prove that a sentence interprets the source correctly. Suggestions need human review. Five simple synthetic extraction examples are only a basic software check, not proof the tool understands real plans.

- 2 MB upload limit and 80,000 extracted character limit in the prototype.
- No cloud LLM is configured or required.
- An assessment is not a readiness score or an approved recovery plan.

In plain terms: AI can point to a possible link; an accountable service owner decides whether it belongs in the verified graph.

06 · ARCHITECTURE & DEPLOYMENT

A conventional, self-hostable application stack

The MVP deliberately keeps the analysis core independent from web, persistence, and AI code.

The interface uses React and TypeScript; Cytoscape draws and lets users edit the network. FastAPI and Pydantic provide and check the API fields. NetworkX checks and calculates the graph. SQLAlchemy saves records in PostgreSQL. Local development and tests can use SQLite. Docker Compose starts the web app, API, and database. Ollama is optional and can run a language model on the same machine or local network.

For the current 43-node demo, PostgreSQL plus NetworkX is simpler than adding a separate graph database. Larger deployments, multiple customer groups, graph history, or time-based records may change that choice; those needs should be confirmed with users first.

Uploaded source documents are read in memory and not retained as files. Extracted quotes and references may be saved with a proposed relationship. This development API has no user login or permissions; it accepts only explicitly fictional nodes. Do not expose it publicly or use real infrastructure data with it.

A customer-ready system would need user accounts, permission levels for editing and approving, separation between customers, secret handling, encrypted connections, backups, data-retention rules, upload scanning, audit monitoring, recovery plans, and a security review. An on-premises option may help a customer control data but increases setup and support work.

- PostgreSQL saves the demo data; SQLite is available for local development.
- No cloud AI is required. A local Ollama model is optional.
- Pinned software versions, license notes, compatibility, and security checks are listed in docs/SOURCES.md.

> See ContinuityMap_Architecture.svg for the picture and ContinuityMap_Technical_Design.md for developer details and API routes.

07 · EVALUATION & IMPACT

Measure the implementation, then test with real owners

Current evaluation checks deterministic software behavior against fixed synthetic fixtures.

The automated checks use small example networks and the 43-item fictional graph. They check backup rules, cycles, disconnected services, missing references, simultaneous disruptions, open questions, conflicting claims, document suggestions, imports, API behavior, and PDF reports. The evaluation also asks whether the app extracts simple examples, keeps human review states straight, uses verified relationships only, preserves uncertainty, points to existing evidence IDs, and includes required report sections.

The latest snapshot shows 5 of 5 simple extraction examples matched, all 3 scenario result lists matched their separately prepared test answers, no unverified relationship caused a service to be marked unavailable, and the required report sections were present. Precision means “of the links suggested, how many were expected?” Recall means “of the expected links, how many were found?” These results describe a few fixed synthetic examples, not real customer documents or real-world disruption accuracy.

Possible customer benefits include less repeated manual scenario preparation, quicker evidence review, earlier visibility of missing backup documentation, better coordination, and clearer records of who reviewed what. These are ideas to test, not proven savings. A pilot could measure preparation time, how many important relationships have current owners and evidence, whether owners agree with the paths, and whether follow-up actions are completed.

The software cannot repair an incomplete or disputed graph on its own. A customer would need to map their own terms and records, assign accountable owners, decide what counts as verified, and schedule regular review.

- Automated suite snapshot: 29 tests passed in the recorded implementation run.
- Graph fixture: 43 nodes, 80 edge records, 78 verified edges, 2 explicit unknown questions.
- Median local calculation time: about 0.04 ms in one test environment; hardware-specific and not a production benchmark.

> The scores show that the software matches its small, hand-checked test cases. They do not show that the fictional graph is complete or that a real service would fail.

08 · BUSINESS CASE, RISKS & SOURCES

Commercial path and responsible next steps

Implementation quality and customer data readiness are central to both value and gross margin.

Here is one illustrative pricing example, not a quote or forecast. Charge $65,000 per year for the organization plus $9,000 per site for three sites: $65,000 + (3 × $9,000) = $92,000 in recurring annual revenue. Add $28,000 for initial onboarding, so first-year revenue is $120,000. Assume direct delivery costs of $18,000 for onboarding plus $12,000 for one year of cloud and support: $30,000 total. First-year gross profit is $120,000 – $30,000 = $90,000, or 75% of revenue. Gross profit leaves out research and development, sales, management, tax, security assurance, and other company costs. If year two costs only $12,000 to host and support, recurring gross margin would be 87%; this too is an assumption.

Possible ways to charge include an organization subscription, a fee per site, an annual customer-hosted license, data-integration work, optional scenario modules, and professional dependency assessments. A broad estimate of time spent preparing exercises suggests $7,200 to $134,400 in possible annual staff-time value, depending on hours, exercise frequency, and staff cost. None of those inputs has been checked with buyers. At the illustrative middle case ($32,000), staff-time savings alone are less than the assumed $92,000 annual subscription. The product may need to show additional value from multi-site coordination, evidence traceability, audits, or consultant time.

Main risks include incomplete or disputed dependency data, sensitive records, AI mistakes, misunderstood results, stale owners, long procurement, and costly customer-specific setup. Start with a narrow approved dataset, named owners, one reviewed exercise, and a controlled deployment. Do not claim to predict outages or certify compliance.

Possible roadmap: months 0–3, interview planners and confirm a safe use case; months 3–6, run a customer-controlled pilot with human-reviewed data and basic identity controls; months 6–12, add approved integrations and stronger access, audit, and security features. Continue only if owners trust the evidence paths and setup effort can be repeated.

- Data completeness and integration effort are the commercial gating factors.
- Everbridge and other established resilience platforms create expected feature overlap; validate fit in buyer-led comparisons.
- The best first pilot is small, has named owners, and uses evidence the team can review.

All prices, costs, and labor values are assumptions for discussion. This is a product concept, not a financial forecast or a legal, safety, or compliance certification.